

**LAPORAN AKHIR PRAKTIKUM**  
**ALGORITMA PEMROGRAMAN DAN STRUKTUR DATA**



Oleh:

Nisa Aulia Kinanti / 124250208

**PROGRAM STUDI SISTEM INFORMASI**  
**JURUSAN INFORMATIKA**  
**FAKULTAS TEKNIK INDUSTRI**  
**UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"**  
**YOGYAKARTA**  
**2026**

HALAMAN PENGESAHAN

LAPORAN AKHIR

Disusun oleh:

Nisa Aulia Kinanti

124250208

Telah Diperiksa dan Disetujui oleh Asisten Praktikum  
Algoritma Pemrograman dan Struktur Data  
Pada Tanggal: 23 Mei 2026

Menyetujui,

Asisten Praktikum

Asisten Praktikum

Constantine Rylianno

Sumakul

124240139

M Dimas Ragil

124240146

|

**KATA PENGANTAR**

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan praktikum Dasar-Dasar Pemrograman serta laporan akhir praktikum Algoritma Pemrograman dan Struktur Data. Adapun laporan ini berisi tentang kumpulan tugas dan evaluasi dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terima kasih kepada asisten dosen yang selalu membimbing dan mengajari kami dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun kami harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, kami ucapkan terima kasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 15 Mei 2026

Penyusun

DAFTAR ISI

HALAMAN PENGESAHAN ..... ii

KATA PENGANTAR ..... iii

DAFTAR ISI .....iv

BAB I TUGAS 1 .....1

1.1 Source Code Program.....1

1.2 Implementasi Materi .....7

1.3 Catatan Revisi ..... 11

1.4 Revisi Program..... 11

1.5 Screenshot Program..... 11

BAB II ARRAY DAN STRING .....14

2.1 Source Code Program..... 14

2.3 Implementasi Materi .....25

2.2 Catatan Revisi .....27

2.3 Revisi Program.....27

2.4 Screenshot Program.....27

BAB III PROYEK AKHIR .....28

3.1 Dasar Teori .....28

3.2 Source Code Program.....30

3.3 Implementasi Materi .....47

3.4 Screenshot Program.....50

3.5 Jadwal Pengerjaan .....53

3.6 Pembagian Tugas .....53

DAFTAR PUSTAKA .....54

**BAB I**  
**TUGAS 1**

Pada praktikum ini dibuat sebuah program pengelolaan data barang elektronik menggunakan bahasa pemrograman C++. Program memanfaatkan konsep *pointer* dan *dynamic array* untuk menyimpan data barang yang terdiri atas ID Barang, Nama Barang, dan Harga Barang. Data disimpan secara permanen pada berkas teks barang.txt menggunakan mekanisme *file handling*. Selain itu, program mengimplementasikan algoritma *Quick Sort* untuk mengurutkan data berdasarkan ID Barang secara menaik (*ascending*) serta algoritma *Binary Search* untuk melakukan pencarian dan penghapusan data secara lebih efisien. Penerapan kedua algoritma tersebut bertujuan untuk meningkatkan efisiensi pengolahan data sesuai dengan konsep struktur data dan algoritma yang dipelajari dalam praktikum. Menurut Cormen et al. (2022), *Quick Sort* merupakan algoritma pengurutan yang memiliki performa rata-rata sangat baik, sedangkan *Binary Search* merupakan metode pencarian yang efisien pada data yang telah terurut.

**1.1 Source Code Program**

```
#include <iostream>
#include <fstream>
using namespace std;

struct Barang {
    int id;
    string nama;
    int harga;
};

//Quick Sort
void quickSort(Barang *arr, int low, int high) {
    int i = low, j = high;
    int pivot = arr[(low + high) / 2].id;

    while (i <= j) {
        while (arr[i].id < pivot) i++;
        while (arr[j].id > pivot) j--;

        if (i <= j) {
            swap(arr[i], arr[j]);
            i++; j--;
        }
    }
}
```

```

    }
}
if (low < j) quickSort(arr, low, j);
if (i < high) quickSort(arr, i, high);
}

//Binary search
int binarySearch(Barang *arr, int n, int key) {
    int kiri = 0, kanan = n - 1;

    while (kiri <= kanan) {
        int mid = (kiri + kanan) / 2;

        if (arr[mid].id == key)
            return mid;
        else if (arr[mid].id < key)
            kiri = mid + 1;
        else
            kanan = mid - 1;
    }
    return -1;
}

//Hitung data
int hitungData() {
    ifstream file("barang.txt");
    Barang b;
    int count = 0;

    while (file >> b.id >> b.nama >> b.harga) {
        count++;
    }
    file.close();
    return count;
}

//Baca file
Barang* bacaFile(int &n) {

```

```

    n = hitungData();
    if (n == 0) return NULL;

    Barang *arr = new Barang[n];
    ifstream file("barang.txt");

    for (int i = 0; i < n; i++) {
        file >> arr[i].id >> arr[i].nama >> arr[i].harga;
    }
    file.close();
    return arr;
}

//simpan file
void simpanFile(Barang *arr, int n) {
    ofstream file("barang.txt");

    for (int i = 0; i < n; i++) {
        file << arr[i].id << " " << arr[i].nama << " " <<
arr[i].harga << endl;
    }
    file.close();
}

//Menu 1
void inputData() {
    ofstream file("barang.txt", ios::app);
    int n;

    cout << "Jumlah data : ";
    cin >> n;

    Barang *arr = new Barang[n];

    for (int i = 0; i < n; i++) {
        cout << "\nID Barang : ";
        cin >> arr[i].id;
        cout << "Nama Barang : ";

```

```

        cin >> arr[i].nama;
        cout << "Harga : ";
        cin >> arr[i].harga;

        file << arr[i].id << " " << arr[i].nama << " " <<
arr[i].harga << endl;
    }
    delete[] arr;
    file.close();
}

//Menu 2
void tampilData() {
    int n;
    Barang *arr = bacaFile(n);

    if (arr == NULL) {
        cout << "Data masih kosong\n";
        return;
    }
    cout << "\n=====Data Barang Elektronik=====\\n";
    for (int i = 0; i < n; i++) {
        cout << "ID : " << arr[i].id
            << ", Nama : " << arr[i].nama
            << ", Harga : " << arr[i].harga << endl;
    }
    delete[] arr;
}

//Menu 3
void sortingData() {
    int n;
    Barang *arr = bacaFile(n);

    if (arr == NULL) {
        cout << "\\nData masih kosong\\n";
        return;
    }
}

```



```

        quickSort(arr, 0, n - 1);
        simpanFile(arr, n);

        cout << "\nData setelah sorting\n";
        for (int i = 0; i < n; i++) {
            cout << "ID : " << arr[i].id
                << ", Nama : " << arr[i].nama
                << ", Harga : " << arr[i].harga << endl;
        }
        delete[] arr;
    }

//Menu 4
void cariData() {
    int n;
    Barang *arr = bacaFile(n);

    if (arr == NULL) {
        cout << "\nData masih kosong\n";
        return;
    }

    int id;
    cout << "Masukkan ID yang dicari : ";
    cin >> id;

    int index = binarySearch(arr, n, id);

    if (index != -1) {
        cout << "\nData ditemukan\n";
        cout << "ID : " << arr[index].id
            << ", Nama : " << arr[index].nama
            << ", Harga : " << arr[index].harga << endl;
    } else {
        cout << "Data tidak ditemukan\n";
    }
    delete[] arr;
}

```

```

}

//Menu 5
void hapusData() {
    int n;
    Barang *arr = bacaFile(n);

    if (arr == NULL) {
        cout << "\nData masih kosong\n";
        return;
    }
    int id;
    cout << "Masukkan ID yang akan dihapus : ";
    cin >> id;

    int index = binarySearch(arr, n, id);

    if (index == -1) {
        cout << "Data tidak ditemukan\n";
    } else {
        for (int i = index; i < n - 1; i++) {
            arr[i] = arr[i + 1];
        }
        simpanFile(arr, n - 1);
        cout << "Data berhasil dihapus\n";
    }
    delete[] arr;
}

//Main
int main() {
    int pilih;

    do {
        cout << "\n===== MENU =====\n";
        cout << "1. Input Data Barang\n";
        cout << "2. Tampilkan Data Barang\n";
        cout << "3. Urutkan Data (Quick Sort)\n";
    }

```

```
cout << "4. Cari Data (Binary Search)\n";
cout << "5. Hapus Data\n";
cout << "0. Keluar\n";
cout << "Pilih menu : ";
cin >> pilih;

switch (pilih) {
    case 1: inputData(); break;
    case 2: tampilData(); break;
    case 3: sortingData(); break;
    case 4: cariData(); break;
    case 5: hapusData(); break;
}
} while (pilih != 0);
}
```

## 1.2 Implementasi Materi

```
Barang *arr = new Barang[n];
```

Kode tersebut mengimplementasikan konsep *dynamic array* menggunakan pointer. Jumlah data barang yang dimasukkan pengguna dapat berbeda-beda sehingga alokasi memori dilakukan secara dinamis menggunakan operator new. Dengan cara ini, program dapat menyesuaikan kebutuhan memori sesuai jumlah data yang dimasukkan tanpa harus menentukan ukuran array secara tetap pada saat kompilasi.

```
void quickSort(Barang *arr, int low, int high) {
    int i = low, j = high;
    int pivot = arr[(low + high) / 2].id;

    while (i <= j) {
        while (arr[i].id < pivot) i++;
        while (arr[j].id > pivot) j--;

        if (i <= j) {
            swap(arr[i], arr[j]);
            i++;
            j--;
        }
    }
}
```

```
    if (low < j)
        quickSort(arr, low, j);

    if (i < high)
        quickSort(arr, i, high);
}
```

Fungsi `quickSort()` digunakan untuk mengurutkan data barang berdasarkan ID Barang secara menaik (*ascending*). Algoritma ini bekerja dengan memilih sebuah nilai sebagai *pivot*, kemudian membagi data menjadi dua bagian, yaitu data yang lebih kecil dari *pivot* dan data yang lebih besar dari *pivot*. Proses tersebut dilakukan secara rekursif hingga seluruh data tersusun secara teratur. Penggunaan *Quick Sort* membuat proses pengurutan menjadi lebih efisien dibandingkan metode pengurutan sederhana pada jumlah data yang besar.

```
int binarySearch(Barang *arr, int n, int key) {
    int kiri = 0, kanan = n - 1;

    while (kiri <= kanan) {
        int mid = (kiri + kanan) / 2;

        if (arr[mid].id == key)
            return mid;
        else if (arr[mid].id < key)
            kiri = mid + 1;
        else
            kanan = mid - 1;
    }

    return -1;
}
```

Fungsi `binarySearch()` digunakan untuk mencari data barang berdasarkan ID Barang. Algoritma ini bekerja dengan membandingkan nilai yang dicari terhadap elemen tengah array yang telah terurut. Jika nilai tidak ditemukan pada bagian tertentu, ruang pencarian akan dipersempit hingga data ditemukan atau seluruh kemungkinan telah diperiksa. Metode ini dipilih karena memiliki efisiensi pencarian yang lebih baik dibandingkan pencarian linear.

```
int hitungData() {
    ifstream file("barang.txt");
    Barang b;
```

```

    int count = 0;

    while (file >> b.id >> b.nama >> b.harga) {
        count++;
    }

    file.close();
    return count;
}

```

Fungsi `hitungData()` digunakan untuk menghitung jumlah data barang yang tersimpan pada berkas `barang.txt`. Jumlah data tersebut diperlukan untuk menentukan ukuran *dynamic array* yang akan dialokasikan ketika data dibaca dari file. Dengan demikian, program dapat mengalokasikan memori sesuai jumlah data yang tersedia.

```

Barang* bacaFile(int &n) {
    n = hitungData();

    if (n == 0)
        return NULL;

    Barang *arr = new Barang[n];
    ifstream file("barang.txt");

    for (int i = 0; i < n; i++) {
        file >> arr[i].id >> arr[i].nama >> arr[i].harga;
    }

    file.close();
    return arr;
}

```

Fungsi `bacaFile()` digunakan untuk membaca seluruh data barang dari berkas teks dan menyimpannya ke dalam *dynamic array*. Implementasi ini menunjukkan penggunaan pointer sebagai media penyimpanan data yang dibaca dari file. Selain itu, fungsi juga melakukan pemeriksaan apabila file belum memiliki data sehingga dapat mencegah terjadinya kesalahan saat proses pembacaan.

```

void simpanFile(Barang *arr, int n) {
    ofstream file("barang.txt");

    for (int i = 0; i < n; i++) {

```

```

        file << arr[i].id << " "
            << arr[i].nama << " "
            << arr[i].harga << endl;
    }

    file.close();
}

```

Fungsi `simpanFile()` digunakan untuk menyimpan data barang dari array ke dalam berkas teks `barang.txt`. Fungsi ini berperan penting setelah proses pengurutan maupun penghapusan data sehingga perubahan yang dilakukan pada data dapat tersimpan secara permanen dan dapat digunakan kembali saat program dijalankan berikutnya.

```

void tampilData() {
    int n;
    Barang *arr = bacaFile(n);

    if (arr == NULL) {
        cout << "Data masih kosong\n";
        return;
    }

    ...
}

```

Menu tampil data digunakan untuk menampilkan seluruh data barang yang tersimpan pada file. Sebelum data ditampilkan, program terlebih dahulu membaca data ke dalam array dinamis. Pada bagian ini juga diterapkan *error handling* untuk menangani kondisi ketika file belum memiliki data sehingga program dapat memberikan informasi yang sesuai kepada pengguna.

```

quickSort(arr, 0, n - 1);
simpanFile(arr, n);

```

Implementasi ini menunjukkan penggunaan algoritma *Quick Sort* pada menu pengurutan data. Setelah data berhasil diurutkan berdasarkan ID Barang, hasil pengurutan disimpan kembali ke dalam file agar urutan data tetap terjaga pada penggunaan program berikutnya. Dengan demikian, data siap digunakan untuk proses pencarian menggunakan *Binary Search*.

```

int index = binarySearch(arr, n, id);

if (index != -1) {
    ...
}

```

```
} else {  
    cout << "Data tidak ditemukan\n";  
}
```

Menu pencarian memanfaatkan algoritma *Binary Search* untuk menemukan data barang berdasarkan ID Barang yang dimasukkan pengguna. Selain melakukan pencarian, program juga menerapkan *error handling* apabila ID yang dicari tidak ditemukan sehingga pengguna memperoleh informasi yang jelas mengenai hasil pencarian.

```
int index = binarySearch(arr, n, id);  
  
if (index == -1) {  
    cout << "Data tidak ditemukan\n";  
} else {  
    for (int i = index; i < n - 1; i++) {  
        arr[i] = arr[i + 1];  
    }  
  
    simpanFile(arr, n - 1);  
}
```

Menu penghapusan data memanfaatkan algoritma Binary Search untuk menemukan posisi data yang akan dihapus. Setelah data ditemukan, seluruh elemen setelah posisi tersebut digeser satu langkah ke depan sehingga data yang dipilih tidak lagi tersimpan dalam array. Hasil perubahan kemudian disimpan kembali ke dalam file agar data yang dihapus tidak muncul pada penggunaan program berikutnya.

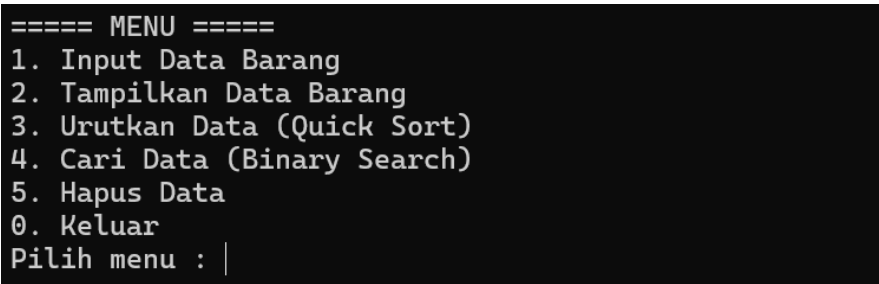
1.3 Catatan Revisi

Tidak ada revisi.

1.4 Revisi Program

```
-
```

1.5 Screenshot Program



Gambar 1.5.1 Tampilan menu

```
Pilih menu : 1
Jumlah data : 3

ID Barang : 4
Nama Barang : Tissue
Harga : 4000

ID Barang : 2
Nama Barang : Gelas
Harga : 25000

ID Barang : 6
Nama Barang : Masker
Harga : 7000
```

Gambar 1.5.2 Menu *input* data

```
Pilih menu : 2

=====Data Barang Elektronik=====
ID : 3, Nama : Casing, Harga : 20000
ID : 4, Nama : Tissue, Harga : 4000
ID : 2, Nama : Gelas, Harga : 25000
ID : 6, Nama : Masker, Harga : 7000
```

Gambar 1.5.3 Menu tampil data

```
Pilih menu : 3

Data setelah sorting
ID : 2, Nama : Gelas, Harga : 25000
ID : 3, Nama : Casing, Harga : 20000
ID : 4, Nama : Tissue, Harga : 4000
ID : 6, Nama : Masker, Harga : 7000
```

Gambar 1.5.4 Menu urutkan data

```
Pilih menu : 4
Masukkan ID yang dicari : 3

Data ditemukan
ID : 3, Nama : Casing, Harga : 20000
```

Gambar 1.5.5 Menu cari data

```
Pilih menu : 5
Masukkan ID yang akan dihapus : 2
Data berhasil dihapus
```

Gambar 1.5.6 Menu hapus data



Pilih menu : 4  
Masukkan ID yang dicari : 5  
Data tidak ditemukan

Gambar 1.5.7 *Error handling* jika id tidak ditemukan

Pilih menu : 2  
Data masih kosong

Gambar 1.5.8 *Error handling* jika data dari file masih kosong

## BAB II

### ARRAY DAN STRING

Pada praktikum ini dibuat sebuah program Sistem Manajemen Antrean Klinik menggunakan bahasa pemrograman C++. Program mengimplementasikan berbagai materi struktur data, yaitu *Binary Search Tree* (BST) untuk penyimpanan data pasien, *Queue* untuk mengelola antrean pemeriksaan, *Stack* untuk fitur *undo*, serta *Array* yang direpresentasikan melalui *vector* untuk kebutuhan pengurutan data. Selain itu, program menerapkan algoritma *Bubble Sort* untuk mengurutkan pasien berdasarkan nama dan *Selection Sort* untuk mengurutkan pasien berdasarkan ID. Data pasien yang dikelola terdiri atas ID Pasien, Nama Pasien, dan Status Pemeriksaan. Implementasi berbagai struktur data tersebut bertujuan untuk memahami karakteristik dan fungsi masing-masing struktur data dalam menyelesaikan permasalahan nyata secara efisien. Menurut Weiss (2014), pemilihan struktur data yang tepat dapat meningkatkan efisiensi penyimpanan, pencarian, dan pengolahan data dalam suatu sistem.

#### 2.1 Source Code Program

```
#include <iostream>
#include <queue>
#include <stack>
#include <iomanip>

using namespace std;

struct Pasien {
    int id;
    string nama;
    string status;
};

struct Node {
    Pasien data;
    Node* left;
    Node* right;

    Node(Pasien p) {
        data = p;
        left = NULL;
        right = NULL;
    }
};
```

```

    }
};

class BST {
private:
    Node* root;

    Node* insert(Node* node, Pasien p) {
        if (node == NULL) {
            return new Node(p);
        }

        if (p.id < node->data.id) {
            node->left = insert(node->left, p);
        } else if (p.id > node->data.id) {
            node->right = insert(node->right, p);
        }

        return node;
    }

    void inorder(Node* node) {
        if (node != NULL) {
            inorder(node->left);
            cout << "ID: " << node->data.id
                 << " | Nama: " << node->data.nama
                 << " | Status: " << node->data.status << endl;
            inorder(node->right);
        }
    }

    Node* search(Node* node, int id) {
        if (node == NULL || node->data.id == id) {
            return node;
        }

        if (id < node->data.id) {
            return search(node->left, id);
        }
    }
};

```

```

    }

    return search(node->right, id);
}

Node* findMin(Node* node) {
    while (node && node->left != NULL) {
        node = node->left;
    }
    return node;
}

Node* deleteNode(Node* node, int id) {
    if (node == NULL) {
        return node;
    }

    if (id < node->data.id) {
        node->left = deleteNode(node->left, id);
    } else if (id > node->data.id) {
        node->right = deleteNode(node->right, id);
    } else {
        if (node->left == NULL) {
            Node* temp = node->right;
            delete node;
            return temp;
        } else if (node->right == NULL) {
            Node* temp = node->left;
            delete node;
            return temp;
        }

        Node* temp = findMin(node->right);
        node->data = temp->data;
        node->right = deleteNode(node->right, temp-
>data.id);
    }
}

```

```

        return node;
    }

    void collectData(Node* node, vector<Pasien>& daftar) {
        if (node != NULL) {
            collectData(node->left, daftar);
            daftar.push_back(node->data);
            collectData(node->right, daftar);
        }
    }

public:
    BST() {
        root = NULL;
    }

    void tambahPasien(Pasien p) {
        if (cariPasien(p.id) != NULL) {
            cout << "ID sudah digunakan!\n";
            return;
        }

        root = insert(root, p);
        cout << "Pasien berhasil ditambahkan ke BST.\n";
    }

    void tampilkanPasien() {
        if (root == NULL) {
            cout << "Data pasien kosong.\n";
            return;
        }

        cout << "\n=== DAFTAR PASIEN (INORDER BST) ===\n";
        inorder(root);
    }

    Node* cariPasien(int id) {
        return search(root, id);
    }

```

```

    }

    void hapusPasien(int id) {
        if (cariPasien(id) == NULL) {
            cout << "Data pasien tidak ditemukan.\n";
            return;
        }

        root = deleteNode(root, id);
        cout << "Data pasien berhasil dihapus.\n";
    }

    vector<Pasien> ambilSemuaData() {
        vector<Pasien> daftar;
        collectData(root, daftar);
        return daftar;
    }
};

queue<int> antrean;
stack<Pasien> undoStack;

void tambahKeAntrean(BST& bst, int id) {
    Node* pasien = bst.cariPasien(id);

    if (pasien == NULL) {
        cout << "Pasien tidak ditemukan.\n";
        return;
    }

    if (pasien->data.status == "Menunggu") {
        antrean.push(id);
        cout << pasien->data.nama << " masuk ke antrean.\n";
    } else {
        cout << "Pasien tidak dapat masuk antrean karena status
bukan 'Menunggu'.\n";
    }
}

```

```

void panggilPasien(BST& bst) {
    if (antrean.empty()) {
        cout << "Antrean kosong.\n";
        return;
    }

    int id = antrean.front();
    antrean.pop();

    Node* pasien = bst.cariPasien(id);

    if (pasien != NULL) {
        undoStack.push(pasien->data);
        pasien->data.status = "Sedang Diperiksa";

        cout << "Pasien " << pasien->data.nama
              << " dipanggil untuk diperiksa.\n";
    }
}

void selesaikanPemeriksaan(BST& bst, int id) {
    Node* pasien = bst.cariPasien(id);

    if (pasien == NULL) {
        cout << "Pasien tidak ditemukan.\n";
        return;
    }

    pasien->data.status = "Selesai";
    cout << "Status pasien berhasil diubah menjadi Selesai.\n";
}

void undoAksiTerakhir(BST& bst) {
    if (undoStack.empty()) {
        cout << "Tidak ada aksi untuk di-undo.\n";
        return;
    }
}

```

```

    Pasien terakhir = undoStack.top();
    undoStack.pop();

    Node* pasien = bst.cariPasien(terakhir.id);

    if (pasien != NULL) {
        pasien->data.status = terakhir.status;
        cout << "Undo berhasil. Status pasien kembali menjadi '"
              << terakhir.status << "'.\n";
    }
}

void lihatAntrean(BST& bst) {
    if (antrean.empty()) {
        cout << "Antrean kosong.\n";
        return;
    }

    queue<int> temp = antrean;

    cout << "\n=== DAFTAR ANTREAN ===\n";

    while (!temp.empty()) {
        int id = temp.front();
        temp.pop();

        Node* pasien = bst.cariPasien(id);

        if (pasien != NULL) {
            cout << "ID: " << pasien->data.id
                  << " | Nama: " << pasien->data.nama
                  << " | Status: " << pasien->data.status << endl;
        }
    }
}

void bubbleSortNama(vector<Pasien>& data) {

```



```

    int n = data.size();

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (data[j].nama > data[j + 1].nama) {
                swap(data[j], data[j + 1]);
            }
        }
    }
}

void selectionSortID(vector<Pasien>& data) {
    int n = data.size();

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < n; j++) {
            if (data[j].id < data[minIndex].id) {
                minIndex = j;
            }
        }

        swap(data[i], data[minIndex]);
    }
}

void tampilkanHasilSort(vector<Pasien> data) {
    for (Pasien p : data) {
        cout << "ID: " << p.id
            << " | Nama: " << p.nama
            << " | Status: " << p.status << endl;
    }
}

void menu() {
    cout << "\n=====\\n";
    cout << " SISTEM MANAJEMEN ANTREAN KLINIK\\n";
}

```

```

    cout << "=====\n";
    cout << "1. Tambah Pasien Baru\n";
    cout << "2. Tampilkan Daftar Pasien\n";
    cout << "3. Tambahkan ke Antrean\n";
    cout << "4. Panggil Pasien Berikutnya\n";
    cout << "5. Selesaikan Pemeriksaan\n";
    cout << "6. Urutkan Data Pasien (Bubble Sort Nama)\n";
    cout << "7. Urutkan Data Pasien (Selection Sort ID)\n";
    cout << "8. Undo Aksi Terakhir\n";
    cout << "9. Lihat Antrean Saat Ini\n";
    cout << "10. Hapus Data Pasien\n";
    cout << "0. Keluar\n";
    cout << "Pilih menu: ";
}

int main() {
    BST bst;
    int pilihan;

    do {
        menu();
        cin >> pilihan;
        cin.ignore();

        switch (pilihan) {
            case 1: {
                Pasien p;

                cout << "Masukkan ID pasien: ";
                cin >> p.id;
                cin.ignore();

                cout << "Masukkan nama pasien: ";
                getline(cin, p.nama);

                p.status = "Menunggu";

                bst.tambahPasien(p);
            }

```

```

        break;
    }

    case 2:
        bst.tampilkanPasien();
        break;

    case 3: {
        int id;
        cout << "Masukkan ID pasien yang ingin masuk antrean:
";
        cin >> id;

        tambahKeAntrean(bst, id);
        break;
    }

    case 4:
        panggilPasien(bst);
        break;

    case 5: {
        int id;
        cout << "Masukkan ID pasien yang selesai diperiksa:
";
        cin >> id;

        selesaikanPemeriksaan(bst, id);
        break;
    }

    case 6: {
        vector<Pasien> data = bst.ambilSemuaData();
        bubbleSortNama(data);

        cout << "\\n=== HASIL BUBBLE SORT BERDASARKAN NAMA
===\\n";
        tampilkanHasilSort(data);
    }

```

```

        break;
    }

    case 7: {
        vector<Pasi en> data = bst.ambilSemuaData();
        selectionSortID(data);

        cout << "\n=== HASIL SELECTION SORT BERDASARKAN ID
===\n";

        tampilkanHasilSort(data);
        break;
    }

    case 8:
        undoAksiTerakhir(bst);
        break;

    case 9:
        lihatAntrean(bst);
        break;

    case 10: {
        int id;
        cout << "Masukkan ID pasien yang ingin dihapus: ";
        cin >> id;

        bst.hapusPasien(id);
        break;
    }

    case 0:
        cout << "Program selesai.\n";
        break;

    default:
        cout << "Pilihan tidak valid.\n";
    }

```

```
    } while (pilihan != 0);  
  
}
```

2.3 Implementasi Materi

```
struct Node {  
    Pasien data;  
    Node* left;  
    Node* right;  
};
```

Potongan kode di atas digunakan untuk membentuk node pada *Binary Search Tree* (BST). Setiap node menyimpan data pasien serta pointer ke anak kiri dan anak kanan yang digunakan untuk membangun struktur pohon.

```
Node* insert(Node* node, Pasien p) {  
    if (node == NULL) {  
        return new Node(p);  
    }  
  
    if (p.id < node->data.id) {  
        node->left = insert(node->left, p);  
    } else if (p.id > node->data.id) {  
        node->right = insert(node->right, p);  
    }  
  
    return node;  
}
```

Potongan kode di atas digunakan untuk menambahkan data pasien ke dalam BST. Posisi penyimpanan data ditentukan berdasarkan nilai ID pasien sehingga struktur BST tetap terjaga.

```
void inorder(Node* node) {  
    if (node != NULL) {  
        inorder(node->left);  
  
        cout << node->data.id;  
  
        inorder(node->right);  
    }  
}
```

```
}

```

Potongan kode di atas digunakan untuk menampilkan seluruh data pasien menggunakan metode traversal inorder. Hasil traversal akan menampilkan data pasien berdasarkan urutan ID dari yang terkecil hingga terbesar.

```
Node* search(Node* node, int id) {
    if (node == NULL || node->data.id == id) {
        return node;
    }

    if (id < node->data.id) {
        return search(node->left, id);
    }

    return search(node->right, id);
}

```

Potongan kode di atas digunakan untuk mencari data pasien berdasarkan ID. Proses pencarian dilakukan dengan menelusuri cabang kiri atau kanan BST sesuai nilai ID yang dicari.

```
Node* deleteNode(Node* node, int id) {
    ...
}

```

Potongan kode di atas digunakan untuk menghapus data pasien dari BST. Setelah data ditemukan, node akan dihapus dan struktur BST akan disesuaikan kembali agar tetap valid.

```
queue<int> antrean;

```

Potongan kode di atas digunakan untuk membuat struktur data antrean (*queue*). Queue berfungsi untuk menyimpan urutan pasien yang akan dipanggil untuk pemeriksaan.

```
antrean.push(id);

```

Potongan kode di atas digunakan untuk memasukkan ID pasien ke dalam antrean pemeriksaan. Data yang masuk pertama kali akan diproses lebih dahulu sesuai konsep FIFO (*First In First Out*).

```
int id = antrean.front();
antrean.pop();

```

Potongan kode di atas digunakan untuk mengambil pasien yang berada di urutan paling depan antrean. Setelah dipanggil, data pasien tersebut akan dikeluarkan dari antrean.

```
stack<Pasien> undoStack;

```

Potongan kode di atas digunakan untuk membuat struktur data stack. Stack berfungsi untuk menyimpan riwayat perubahan status pasien yang nantinya dapat digunakan pada fitur *undo*.

```
undoStack.push(pasien->data);
```

Potongan kode di atas digunakan untuk menyimpan data pasien ke dalam stack sebelum status pasien diubah. Data tersebut akan digunakan apabila pengguna melakukan proses *undo*.

```
Pasien terakhir = undoStack.top();  
undoStack.pop();
```

Potongan kode di atas digunakan untuk mengambil data terakhir yang tersimpan pada stack. Data tersebut digunakan untuk mengembalikan status pasien ke kondisi sebelumnya.

```
vector<Pasien> daftar;
```

Potongan kode di atas digunakan untuk membuat *array* dinamis menggunakan vector. Vector berfungsi untuk menampung kumpulan data pasien yang akan diproses atau diurutkan.

```
void bubbleSortNama(vector<Pasien>& data)
```

Potongan kode di atas digunakan untuk mengurutkan data pasien berdasarkan nama. Metode *Bubble Sort* bekerja dengan membandingkan dua data yang berdekatan kemudian menukarnya apabila urutannya tidak sesuai.

```
void selectionSortID(vector<Pasien>& data)
```

Potongan kode di atas digunakan untuk mengurutkan data pasien berdasarkan ID. Metode *Selection Sort* bekerja dengan mencari nilai terkecil kemudian menempatkannya pada posisi yang sesuai.

2.2 Catatan Revisi

Tidak ada revisi.

2.3 Revisi Program

```
-
```

2.4 Screenshot Program

-

**BAB III**  
**PROYEK AKHIR**

Pada praktikum proyek akhir ini dibuat sebuah program Sistem Monitoring Satelit menggunakan bahasa pemrograman C++. Program dirancang untuk membantu pengelolaan data satelit yang sedang dipantau, mulai dari pendaftaran satelit, pengelolaan antrean monitoring, proses analisis satelit, hingga penyimpanan data secara permanen ke dalam berkas. Program mengimplementasikan berbagai materi yang telah dipelajari selama praktikum, yaitu pointer, *Binary Search Tree* (BST), *Queue*, *Stack*, *Searching*, *Sorting*, dan *File Handling*. BST digunakan sebagai struktur utama penyimpanan data satelit berdasarkan ID, queue digunakan untuk mengatur antrean monitoring, stack digunakan untuk mendukung fitur *undo*, sedangkan algoritma *Bubble Sort* dan *Selection Sort* digunakan untuk mengurutkan data satelit. Dengan mengintegrasikan berbagai struktur data dan algoritma tersebut, program mampu mengelola data satelit secara lebih terstruktur dan efisien.

**3.1 Dasar Teori**

**Sistem Monitoring Satelit**

Sistem Monitoring Satelit merupakan sistem yang digunakan untuk mengelola dan memantau kondisi operasional satelit. Sistem ini dapat digunakan untuk mencatat data satelit, memantau status satelit yang sedang dianalisis, mengelola antrean monitoring, serta menyimpan data secara permanen ke dalam berkas. Dalam implementasinya, sistem monitoring satelit memanfaatkan berbagai struktur data dan algoritma seperti *Binary Search Tree* (BST), *Queue*, *Stack*, *Sorting*, dan *File Handling* untuk meningkatkan efisiensi pengolahan data.

**Binary Search Tree (BST)**

*Binary Search Tree* (BST) merupakan struktur data berbentuk pohon biner yang memiliki aturan bahwa setiap node pada subtree kiri memiliki nilai yang lebih kecil dari node induknya, sedangkan setiap node pada subtree kanan memiliki nilai yang lebih besar. BST banyak digunakan untuk proses penyimpanan, pencarian, dan penghapusan data karena mampu mempercepat proses pencarian dibandingkan struktur data linear. Menurut Cormen et al. (2022), BST merupakan salah satu struktur data dasar yang digunakan untuk mendukung operasi pencarian secara efisien.

**Queue**

*Queue* merupakan struktur data linear yang menerapkan prinsip *First In First Out* (FIFO), yaitu data yang pertama masuk akan menjadi data pertama yang diproses atau dikeluarkan. Struktur data ini umumnya digunakan untuk mengelola sistem antrean seperti



antrean pelayanan, antrean proses, maupun antrean monitoring dalam suatu sistem informasi. Menurut Weiss (2014), queue sangat sesuai digunakan pada sistem yang membutuhkan pemrosesan data berdasarkan urutan kedatangan.

### **Stack**

*Stack* merupakan struktur data linear yang menerapkan prinsip *Last In First Out* (LIFO), yaitu data yang terakhir masuk akan menjadi data pertama yang dikeluarkan. Struktur data ini sering digunakan untuk implementasi fitur *undo*, pengelolaan riwayat aktivitas, serta pemrosesan ekspresi dalam pemrograman. Menurut Weiss (2014), stack merupakan salah satu struktur data fundamental yang banyak digunakan dalam pengembangan perangkat lunak.

### **Bubble Sort**

*Bubble Sort* merupakan algoritma pengurutan yang bekerja dengan membandingkan dua elemen yang berdekatan dan menukarnya apabila urutannya tidak sesuai. Proses tersebut dilakukan secara berulang hingga seluruh data tersusun secara teratur. Algoritma ini mudah diimplementasikan dan sering digunakan untuk mempelajari konsep dasar pengurutan data. Menurut Cormen et al. (2022), algoritma pengurutan merupakan bagian penting dalam pengolahan data karena dapat meningkatkan efisiensi proses pencarian dan pengelolaan informasi.

### **Selection Sort**

*Selection Sort* merupakan algoritma pengurutan yang bekerja dengan mencari elemen terkecil dari bagian data yang belum teratur, kemudian menempatkannya pada posisi yang sesuai. Proses ini dilakukan berulang hingga seluruh data tersusun secara teratur. Algoritma ini memiliki konsep yang sederhana sehingga sering digunakan untuk memahami dasar proses pengurutan data.

### **File Handling**

*File Handling* merupakan teknik pengolahan berkas yang digunakan untuk menyimpan dan membaca data dari media penyimpanan. Dengan adanya *file handling*, data yang telah dimasukkan pengguna dapat disimpan secara permanen sehingga tetap tersedia meskipun program telah ditutup. Menurut Deitel dan Deitel (2020), pengelolaan file merupakan salah satu kemampuan penting dalam pemrograman karena memungkinkan data disimpan dan digunakan kembali pada eksekusi program berikutnya.

3.2 Source Code Program

```
#include <iostream>
#include <fstream>
#include <queue>
#include <stack>
#include <vector>
#include <sstream>

using namespace std;

struct Satelit {
    int id;
    string nama;
    string orbit;
    string masalah;
    string status;
};

struct Node {
    Satelit data;
    Node* left;
    Node* right;
};

struct Action {
    string type; // PROCESS / FINISH
    int id;
};

Node* root = NULL;
queue<int> antrian;
stack<Action> riwayat;
vector<int> idDalamQueue;

// ===== BST =====

Node* createNode(int id, string nama, string orbit, string
masalah, string status = "Menunggu Monitoring") {
```

Formatted Table

```

Node* newNode = new Node;
newNode->data.id = id;
newNode->data.nama = nama;
newNode->data.orbit = orbit;
newNode->data.masalah = masalah;
newNode->data.status = status;
newNode->left = NULL;
newNode->right = NULL;
return newNode;
}

Node* search(Node* root, int id) {
    if (root == NULL || root->data.id == id)
        return root;

    if (id < root->data.id)
        return search(root->left, id);

    return search(root->right, id);
}

Node* insertNode(Node* root,
                int id,
                string nama,
                string orbit,
                string masalah,
                string status = "Menunggu Monitoring",
                bool tampilPesan = true) {

    if (root == NULL) {

        if (tampilPesan)
            cout << "\nSatelit berhasil didaftarkan\n";

        return createNode(id, nama, orbit, masalah, status);
    }

    if (id < root->data.id)

```

```

        root->left = insertNode(root->left,
                                id,
                                nama,
                                orbit,
                                masalah,
                                status,
                                tampilPesan);

    else if (id > root->data.id)
        root->right = insertNode(root->right,
                                id,
                                nama,
                                orbit,
                                masalah,
                                status,
                                tampilPesan);

    else
        cout << "\nID sudah digunakan\n";

    return root;
}

void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);

        cout << root->data.id << " | "
              << root->data.nama << " | "
              << root->data.orbit << " | "
              << root->data.masalah << " | "
              << root->data.status << endl;

        inorder(root->right);
    }
}

Node* minValueNode(Node* node) {

```

```

Node* current = node;

while (current && current->left != NULL)
    current = current->left;

return current;
}

Node* deleteNode(Node* root, int id) {
    if (root == NULL)
        return root;

    if (id < root->data.id)
        root->left = deleteNode(root->left, id);

    else if (id > root->data.id)
        root->right = deleteNode(root->right, id);

    else {
        if (root->left == NULL) {
            Node* temp = root->right;
            delete root;
            return temp;
        }

        else if (root->right == NULL) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        Node* temp = minValueNode(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data.id);
    }

    return root;
}

```

```

// ===== TUNGGU =====

void tunggu() {
    cout << "\nTekan ENTER untuk kembali ke menu...";
    cin.ignore();
    cin.get();
}

void tungguTanpaIgnore() {
    cout << "\nTekan ENTER untuk kembali ke menu...";
    cin.get();
}

// ===== QUEUE =====

bool sudahDalamQueue(int id) {
    for (int x : idDalamQueue) {
        if (x == id)
            return true;
    }
    return false;
}

void tambahAntrian() {
    int id;
    cout << "Masukkan ID satelit: ";
    cin >> id;

    Node* satelit = search(root, id);

    if (satelit == NULL) {
        cout << "\nSatelit tidak ditemukan\n";
        return;
    }

    if (sudahDalamQueue(id)) {
        cout << "\nSatelit sudah ada dalam antrian monitoring\n";
    }
}

```

```

        return;
    }

    if (satelit->data.status != "Menunggu Monitoring") {
        cout << "\nSatelit tidak dapat masuk antrian\n";
        return;
    }

    antrian.push(id);
    idDalamQueue.push_back(id);

    cout << "\nSatelit " << satelit->data.nama
        << " ditambahkan ke antrian monitoring\n";
}

void prosesMonitoring() {
    if (antrian.empty()) {
        cout << "\nTidak ada satelit dalam antrian monitoring\n";
        return;
    }

    int id = antrian.front();
    antrian.pop();

    idDalamQueue.erase(idDalamQueue.begin());

    Node* satelit = search(root, id);

    satelit->data.status = "Sedang Dianalisis";

    Action act;
    act.type = "PROCESS";
    act.id = id;

    riwayat.push(act);

    cout << "\nSatelit "
        << satelit->data.nama

```

```

        << " sedang dianalisis\n";
    }

void lihatAntrian() {
    if (antrian.empty()) {
        cout << "Antrian monitoring kosong\n";
        return;
    }

    queue<int> temp = antrian;
    int posisi = 1;

    while (!temp.empty()) {
        int id = temp.front();
        temp.pop();

        Node* satelit = search(root, id);

        cout << posisi << ". "
              << satelit->data.nama
              << " (" << satelit->data.id << ")\n";

        posisi++;
    }
}

// ===== STACK UNDO =====

void selesaiMonitoring() {
    int id;

    cout << "Masukkan ID satelit: ";
    cin >> id;

    Node* satelit = search(root, id);

    if (satelit == NULL) {
        cout << "\nSatelit tidak ditemukan\n";
    }
}

```



```

        return;
    }

    if (satelit->data.status != "Sedang Dianalisis") {
        cout << "\nSatelit belum dalam proses analisis\n";
        return;
    }

    satelit->data.status = "Selesai Monitoring";

    Action act;
    act.type = "FINISH";
    act.id = id;

    riwayat.push(act);

    cout << "\nMonitoring satelit "
         << satelit->data.nama
         << " selesai\n";
}

void undoAksi() {
    if (riwayat.empty()) {
        cout << "\nTidak ada aksi untuk di-undo\n";
        return;
    }

    Action last = riwayat.top();
    riwayat.pop();

    Node* satelit = search(root, last.id);

    if (satelit == NULL) {
        cout << "\nData satelit tidak ditemukan\n";
        return;
    }

    if (last.type == "PROCESS") {

```

```

        satelit->data.status = "Menunggu Monitoring";

        antrian.push(last.id);
        idDalamQueue.push_back(last.id);

        cout << "\nUndo: Satelit "
              << satelit->data.nama
              << " dikembalikan ke antrian monitoring\n";
    }

    else if (last.type == "FINISH") {
        satelit->data.status = "Sedang Dianalisis";

        cout << "\nUndo: Status satelit "
              << satelit->data.nama
              << " dikembalikan ke Sedang Dianalisis\n";
    }
}

// ===== SORTING =====

void simpanData(Node* root, vector<Satelit>& data) {
    if (root != NULL) {
        simpanData(root->left, data);
        data.push_back(root->data);
        simpanData(root->right, data);
    }
}

void bubbleSortNama(vector<Satelit>& data) {
    int n = data.size();

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (data[j].nama > data[j + 1].nama) {
                swap(data[j], data[j + 1]);
            }
        }
    }
}

```

```

    }

    cout << "\nData diurutkan berdasarkan nama satelit\n";

    for (Satelit s : data) {
        cout << s.id << " | "
            << s.nama << " | "
            << s.orbit << " | "
            << s.masalah << " | "
            << s.status << endl;
    }
}

void selectionSortID(vector<Satelit>& data) {
    int n = data.size();

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < n; j++) {
            if (data[j].id < data[minIndex].id) {
                minIndex = j;
            }
        }

        swap(data[i], data[minIndex]);
    }

    cout << "\nData diurutkan berdasarkan ID satelit\n";

    for (Satelit s : data) {
        cout << s.id << " | "
            << s.nama << " | "
            << s.orbit << " | "
            << s.masalah << " | "
            << s.status << endl;
    }
}

```

```
// ===== FILE HANDLING =====

void tulisFile(Node* root, ofstream& file) {
    if (root != NULL) {
        tulisFile(root->left, file);

        file << root->data.id << "|"
            << root->data.nama << "|"
            << root->data.orbit << "|"
            << root->data.masalah << "|"
            << root->data.status << endl;

        tulisFile(root->right, file);
    }
}

void simpanKeFile() {
    ofstream file("data_satelit.txt");

    if (!file) {
        cout << "File gagal dibuka\n";
        return;
    }

    tulisFile(root, file);
    file.close();

    cout << "Data satelit berhasil disimpan ke file\n";
}

void bacaDariFile() {
    ifstream file("data_satelit.txt");

    if (!file) {
        cout << "File data_satelit.txt tidak ditemukan\n";
        return;
    }
}
```

```

    root = NULL;

    string line;

    while (getline(file, line)) {
        stringstream ss(line);
        string idStr, nama, orbit, masalah, status;

        getline(ss, idStr, '|');
        getline(ss, nama, '|');
        getline(ss, orbit, '|');
        getline(ss, masalah, '|');
        getline(ss, status, '|');

        int id = stoi(idStr);

        root = insertNode(root, id, nama, orbit, masalah, status,
false);
    }

    file.close();

    cout << "\nData satelit berhasil dibaca dari file\n";
}

// ===== MAIN =====

int main() {
    system("cls");

    int pilihan;

    do {
        system("cls");

        cout << "\n===== SISTEM MONITORING SATELIT =====\n";
        cout << "1. Daftarkan Satelit\n";

```

```

cout << "2. Tampilkan Data Satelit\n";
cout << "3. Tambah ke Antrian Monitoring\n";
cout << "4. Proses Monitoring Satelit\n";
cout << "5. Selesaikan Monitoring\n";
cout << "6. Sorting Data Satelit\n";
cout << "7. Undo Aksi\n";
cout << "8. Lihat Antrian Monitoring\n";
cout << "9. Hapus Data Satelit\n";
cout << "10. Simpan Data ke File\n";
cout << "11. Baca Data dari File\n";
cout << "0. Keluar\n";
cout << "Pilihan: ";
cin >> pilihan;

switch (pilihan) {

case 1: {
    system("cls");

    cout << "-----\n";
    cout << "1. Daftarkan Satelit\n";
    cout << "-----\n";

    int id;
    string nama, orbit, masalah;

    cout << "ID Satelit: ";
    cin >> id;
    cin.ignore();

    cout << "Nama Satelit: ";
    getline(cin, nama);

    cout << "Jenis Orbit: ";
    getline(cin, orbit);

    cout << "Masalah Satelit: ";
    getline(cin, masalah);

```

```

        root = insertNode(root, id, nama, orbit, masalah);
        tungguTanpaIgnore();
        break;
    }

    case 2:
        system("cls");

        cout << "-----\n";
        cout << "2. Tampilkan Data Satelit\n";
        cout << "-----\n";

        if (root == NULL) {
            cout << "Belum ada data satelit\n";
        } else {
            cout << "ID | Nama | Orbit | Masalah | Status\n";
            cout << "-----\n";
            inorder(root);
        }

        tunggu();
        break;

    case 3:
        system("cls");

        cout << "-----\n";
        cout << "3. Tambah ke Antrian Monitoring\n";
        cout << "-----\n";

        tambahAntrian();
        tunggu();
        break;

    case 4:
        system("cls");

```

```

        cout << "-----\n";
        cout << "4. Proses Monitoring Satelit\n";
        cout << "-----\n";

        prosesMonitoring();
        tunggu();
        break;

    case 5:
        system("cls");

        cout << "-----\n";
        cout << "5. Selesaikan Monitoring\n";
        cout << "-----\n";

        selesaiMonitoring();
        tunggu();
        break;

    case 6: {
        system("cls");

        cout << "-----\n";
        cout << "6. Sorting Data Satelit\n";
        cout << "-----\n";

        vector<Satelit> data;
        simpanData(root, data);

        if (data.empty()) {
            cout << "Belum ada data satelit\n";
            tunggu();
            break;
        }

        int pilihSort;

```



```

        cout << "1. Bubble Sort berdasarkan Nama\n";
        cout << "2. Selection Sort berdasarkan ID\n";
        cout << "Pilihan: ";
        cin >> pilihSort;

        if (pilihSort == 1)
            bubbleSortNama(data);
        else if (pilihSort == 2)
            selectionSortID(data);
        else
            cout << "Pilihan sorting tidak valid\n";

        tunggu();
        break;
    }

    case 7:
        system("cls");

        cout << "-----\n";
        cout << "7. Undo Aksi\n";
        cout << "-----\n";

        undoAksi();
        tunggu();
        break;

    case 8:
        system("cls");

        cout << "-----\n";
        cout << "8. Lihat Antrian Monitoring\n";
        cout << "-----\n";

        lihatAntrian();
        tunggu();
        break;

```

```

case 9: {
    system("cls");

    cout << "-----\n";
    cout << "9. Hapus Data Satelit\n";
    cout << "-----\n";

    int id;

    cout << "Masukkan ID satelit: ";
    cin >> id;

    Node* cek = search(root, id);

    if (cek == NULL) {
        cout << "Satelit tidak ditemukan\n";
    } else {
        root = deleteNode(root, id);
        cout << "Data satelit berhasil dihapus\n";
    }

    tunggu();
    break;
}

case 10:
    system("cls");

    cout << "-----\n";
    cout << "10. Simpan Data ke File\n";
    cout << "-----\n";

    simpanKeFile();
    tunggu();
    break;

case 11:
    system("cls");

```

```
        cout << "-----\n";
        cout << "11. Baca Data dari File\n";
        cout << "-----\n";

        bacaDariFile();
        tunggu();
        break;

    case 0:
        cout << "Program selesai\n";
        break;

    default:
        system("cls");

        cout << "Pilihan tidak valid\n";
        tunggu();
    }

} while (pilihan != 0);

}
```

3.3 Implementasi Materi

```
struct Node {
    Satelit data;
    Node* left;
    Node* right;
};
```

Potongan kode di atas mengimplementasikan konsep pointer pada struktur BST. Pointer left dan right digunakan untuk menghubungkan node dengan anak kiri dan anak kanan sehingga membentuk struktur pohon.

```
Node* insertNode(Node* root,
                 int id,
                 string nama,
                 string orbit,
```

```
string masalah,  
string status = "Menunggu Monitoring")
```

Potongan kode di atas digunakan untuk menambahkan data satelit ke dalam BST berdasarkan ID satelit. Data dengan ID lebih kecil akan ditempatkan pada subtree kiri, sedangkan ID yang lebih besar akan ditempatkan pada subtree kanan.

```
Node* search(Node* root, int id) {  
    if (root == NULL || root->data.id == id)  
        return root;  
  
    if (id < root->data.id)  
        return search(root->left, id);  
  
    return search(root->right, id);  
}
```

Potongan kode di atas digunakan untuk mencari data satelit berdasarkan ID. Proses pencarian dilakukan dengan menelusuri cabang BST yang sesuai hingga data ditemukan.

```
Node* deleteNode(Node* root, int id)
```

Potongan kode di atas digunakan untuk menghapus data satelit dari BST. Setelah data ditemukan, struktur pohon akan disesuaikan kembali agar aturan BST tetap terjaga.

```
void inorder(Node* root) {  
    if (root != NULL) {  
        inorder(root->left);  
        cout << root->data.nama;  
        inorder(root->right);  
    }  
}
```

Potongan kode di atas digunakan untuk menampilkan seluruh data satelit menggunakan traversal inorder sehingga data akan tampil berdasarkan urutan ID secara menaik.

```
queue<int> antrian;
```

Potongan kode di atas digunakan untuk membuat struktur data antrian monitoring satelit. Queue digunakan untuk mengatur urutan satelit yang akan diproses berdasarkan prinsip FIFO (*First In First Out*).

```
antrian.push(id);
```

Potongan kode di atas digunakan untuk memasukkan ID satelit ke dalam antrian monitoring sehingga satelit dapat diproses sesuai urutan kedatangannya.

```
int id = antrian.front();
```

```
antrian.pop();
```

Potongan kode di atas digunakan untuk mengambil satelit yang berada di posisi paling depan antrian dan mengeluarkannya dari queue untuk diproses monitoring.

```
stack<Action> riwayat;
```

Potongan kode di atas digunakan untuk menyimpan riwayat aktivitas monitoring satelit. Struktur data stack digunakan untuk mendukung fitur *undo*.

```
riwayat.push(act);
```

Potongan kode di atas digunakan untuk menyimpan aktivitas terakhir ke dalam stack sehingga dapat dikembalikan apabila pengguna melakukan proses *undo*.

```
Action last = riwayat.top();  
riwayat.pop();
```

Potongan kode di atas digunakan untuk mengambil aktivitas terakhir yang tersimpan pada stack. Data tersebut digunakan untuk mengembalikan kondisi satelit ke status sebelumnya.

```
vector<Satelit> data;
```

Potongan kode di atas digunakan untuk menyimpan kumpulan data satelit secara dinamis sebelum dilakukan proses pengurutan.

```
void bubbleSortNama(vector<Satelit>& data)
```

Potongan kode di atas digunakan untuk mengurutkan data satelit berdasarkan nama satelit secara alfabetis menggunakan metode *Bubble Sort*.

```
void selectionSortID(vector<Satelit>& data)
```

Potongan kode di atas digunakan untuk mengurutkan data satelit berdasarkan ID satelit menggunakan metode *Selection Sort*.

```
ofstream file("data_satelit.txt");
```

Potongan kode di atas digunakan untuk membuka atau membuat file data\_satelit.txt yang digunakan sebagai media penyimpanan data satelit secara permanen.

3.4 Screenshot Program

```
===== SISTEM MONITORING SATELIT =====
1. Daftarkan Satelit
2. Tampilkan Data Satelit
3. Tambah ke Antrian Monitoring
4. Proses Monitoring Satelit
5. Selesaikan Monitoring
6. Sorting Data Satelit
7. Undo Aksi
8. Lihat Antrian Monitoring
9. Hapus Data Satelit
10. Simpan Data ke File
11. Baca Data dari File
0. Keluar
Pilihan: |
```

Gambar 3.4.1 Tampilan awal menu

```
-----
1. Daftarkan Satelit
-----
ID Satelit: 101
Nama Satelit: Nusantara-1
Jenis Orbit: GEO
Masalah Satelit: Gangguan Sinyal

Satelit berhasil didaftarkan

Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.2 Tampilan daftar satelit

```
-----
2. Tampilkan Data Satelit
-----
ID | Nama | Orbit | Masalah | Status
-----
101 | Nusantara-1 | GEO | Gangguan Sinyal | Menunggu Monitoring
102 | Lapan-A2 | LEO | Sensor Suhu Tidak Stabil | Menunggu Monitoring
103 | Telkom-4 | GEO | Gangguan Komunikasi | Menunggu Monitoring

Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.3 Tampilan data satelit

```
-----
3. Tambah ke Antrian Monitoring
-----
Masukkan ID satelit: 103

Satelit Telkom-4 ditambahkan ke antrian monitoring

Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.4 Tampilan tambah antrian monitoring satelit

```
-----
4. Proses Monitoring Satelit
-----

Satelit Telkom-4 sedang dianalisis

Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.5 Tampilan proses monitoring satelit

```
-----
5. Selesaikan Monitoring
-----

Masukkan ID satelit: 103

Monitoring satelit Telkom-4 selesai

Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.6 Tampilan penyelesaian monitoring

```
-----
6. Sorting Data Satelit
-----

1. Bubble Sort berdasarkan Nama
2. Selection Sort berdasarkan ID
Pilihan: 1

Data diurutkan berdasarkan nama satelit
102 | Lapan-A2 | LEO | Sensor Suhu Tidak Stabil | Menunggu Monitoring
101 | Nusantara-1 | GEO | Gangguan Sinyal | Menunggu Monitoring
103 | Telkom-4 | GEO | Gangguan Komunikasi | Selesai Monitoring

Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.7 Tampilan sorting data satelit (Bubble Sort)

```
-----
6. Sorting Data Satelit
-----

1. Bubble Sort berdasarkan Nama
2. Selection Sort berdasarkan ID
Pilihan: 2

Data diurutkan berdasarkan ID satelit
101 | Nusantara-1 | GEO | Gangguan Sinyal | Menunggu Monitoring
102 | Lapan-A2 | LEO | Sensor Suhu Tidak Stabil | Menunggu Monitoring
103 | Telkom-4 | GEO | Gangguan Komunikasi | Selesai Monitoring

Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.8 Tampilan sorting data satelit (Selection Sort)

```
-----
7. Undo Aksi
-----

Undo: Status satelit Telkom-4 dikembalikan ke Sedang Dianalisis

Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.9 Tampilan Undo aksi

```
-----  
8. Lihat Antrian Monitoring  
-----  
1. Nusantara-1 (101)  
  
Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.10 Tampilan lihat antrian monitoring

```
-----  
9. Hapus Data Satelit  
-----  
Masukkan ID satelit: 102  
Data satelit berhasil dihapus  
  
Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.11 Tampilan hapus data satelit

```
-----  
10. Simpan Data ke File  
-----  
Data satelit berhasil disimpan ke file  
  
Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.12 Tampilan simpan data ke file

```
-----  
11. Baca Data dari File  
-----  
  
Data satelit berhasil dibaca dari file  
  
Tekan ENTER untuk kembali ke menu...|
```

Gambar 3.4.13 Tampilan baca data dari file



3.5 Jadwal Pengerjaan

Berikut ini merupakan jadwal singkat dalam pengerjaan proyek akhir “Sistem Monitoring Satelit”, yaitu:

Tabel 3.5.1 Jadwal Pengerjaan Proyek Akhir “Sistem Monitoring Satelit”

| No | Kegiatan                                             | Pengerjaan |    |    |    |    |
|----|------------------------------------------------------|------------|----|----|----|----|
|    |                                                      | Mei        |    |    |    |    |
|    |                                                      | 18         | 19 | 20 | 21 | 22 |
| 1  | Mengkaji Masalah                                     |            |    |    |    |    |
| 2  | Merancang Program                                    |            |    |    |    |    |
| 3  | Membuat <i>Source Code</i>                           |            |    |    |    |    |
| 4  | Implementasi Struktur Data                           |            |    |    |    |    |
| 5  | Implementasi <i>Sorting</i> dan <i>File Handling</i> |            |    |    |    |    |
| 6  | <i>Testing</i> Program                               |            |    |    |    |    |

3.6 Pembagian Tugas

Berikut ini merupakan pembagian tugas selama pengerjaan proyek akhir “Sistem Monitoring Satelit”, yaitu:

Tabel 3.6.1 Pembagian Tugas Proyek Akhir “Sistem Monitoring Satelit”

| No | Kegiatan                                             | Penanggung Jawab       |            |
|----|------------------------------------------------------|------------------------|------------|
|    |                                                      | Haris Saifullah Siddiq | Nisa Aulia |
| 1  | Mengkaji Masalah                                     |                        |            |
| 2  | Merancang Program                                    |                        |            |
| 3  | Membuat <i>Source Code</i>                           |                        |            |
| 4  | Implementasi Struktur Data                           |                        |            |
| 5  | Implementasi <i>Sorting</i> dan <i>File Handling</i> |                        |            |
| 6  | <i>Testing</i> Program                               |                        |            |
| 7  | Penyusunan Laporan                                   |                        |            |

## DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Deitel, P. J., & Deitel, H. M. (2020). *C++ how to program* (10th ed.). Pearson.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Weiss, M. A. (2014). *Data structures and algorithm analysis in C++* (4th ed.). Pearson.